

Multiarm Bandits

Multiarm Bandit היא בעיית ה-RL¹ הכי קטנה שאפשר לדבר עליה, אבל עדיין משתמשים בה המון במחקר ובתעשייה.

- נדבר על: MDP עם מצב יחיד $S = \{s_1\}$
- k פעולות, שכל פעולה a_i לוקחת אותנו מ- s_1 חזרה ל- s_1 .
- התגמול הוא $R(a_i) = R(s_1, a_i, s_1)$
- כלומר תמיד נגיע לאותו מצב, אבל התגמול יכול להיות שונה - וב-Multiarm Bandit התגמול הוא סטוכסטי. אנחנו לא יודעים מה התגמול של כל זרוע, ולכן policy צריך גם לגלות אותו.
- ה-policy π תלוי בזמן:
- אם יש אופק סופי T :

$$V_\pi(s_1) = \mathbb{E} \left[\sum_{t=0}^T R_t(a) \middle| a \in \pi \right]$$

אם האופק אינסופי, אז מסתכלים על התגמול הממוצע:

$$V_\pi(s_1) = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} \left[\sum_{t=0}^T R_t(a) \middle| a \in \pi \right]$$

הבעיה היא ש-rewards לא ידועים - וסטוכסטיים.

- אם ה-rewards היו ידועים - היינו פשוט לוחצים על הכפתור עם תוחלת התגמול הכי גדולה.
 - אם זה היה דטרמיניסטי זה היה מאוד קל - פשוט מנסים כל כפתור פעם אחת כדי לדעת את כל ה-rewards, ואז בוחרים
- השאלה, אם כן, היא מתי יודעים ש"למדנו מספיק". אם היינו יכולים ללחוץ על כל כפתור אינסוף פעמים היינו יכולים ללמוד את ההתפלגויות המדויקות. אבל בפועל זה לא אפשרי.
- פורמלית: K -armed bandit הוא $\langle K, \mathcal{A}, \mathcal{D}, R, T \rangle$:

$$\begin{array}{ll} K \in \mathbb{N} & \text{כמות ה-arms (פעולות)} \\ \mathcal{A} & \text{סט של פעולות, } |\mathcal{A}| = K \\ \mathcal{D} & \text{סט סטציונרי של התפלגויות על } \mathbb{R} \end{array}$$

$$a \in \mathcal{A} \implies D_a \in \mathcal{D}$$

אנחנו לא יודעים את ה- D_a - אנחנו רק יודעים שהם קיימים.

$R(a) \sim D_a$ משתנה מקרי, נדגם באופן בלתי תלוי מ D_a . $R(a)$ הוא חסום.
 $T \in \mathbb{N}$ אופק סופי (מספר צעדים)

- המטרה: • למקסם את $\mathbb{E} \left[\sum_{t=1}^T R(a) \mid a \in \mathcal{A} \right]$ (כלומר למצוא את a הכי טוב)
- לעשות את זה Online - כלומר ההחלטה מה לעשות (איזה a לבחור) בזמן t מתבצעת בזמן $t - 1$.

אלגוריתמים של explore first עובדים בשני שלבים:
• שלב exploration - בוחרים פעולות לא בגלל שהן הכי טובות, אלא כדי לגלות ו/או לבסס עוד ידע.

- שלב exploitation - בוחרים פעולה כי חושבים שהיא הכי טובה.
- ב explore first מחלקים את הזמן t צעדים ראשונים משקיעים ב exploration ואת השאר ב exploitation.
- איך עושים evaluation לאלגוריתמים? משווים לאלגוריתם "הכי טוב". נגדיר את התוחלת האמיתית:

$$\mu(a) := \mathbb{E}_\infty[R(a)] = \mathbb{E}_\infty[D_a]$$

ואת $\mu^* = \max_{a \in \mathcal{A}} \mu(a)$ μ^* התוחלת הכי טובה. לכן התואצה התיאורית האופטימלית היא $\mu^* \cdot T$.

הגדרה: החרטה (regret) היא:

$$\rho(T) := \mu^* \cdot T - \sum_{t=1}^T \mu(a_t)$$

אלגוריתמים טובים מניבים חרטה יותר נמוכה.

אלגוריתם ϵ -first explore first

זהו האלגוריתם הכי פשוט. בוחרים $N = \epsilon T$ צעדים (כלומר אחוז מתוך האופק שלנו) ובזמן הזה עושים round robin על הפעולות בשביל לאסוף עליהן סטטיסטיקה. אחרי זה, נבחר את הפעולה עם הממוצע הכי טוב ונריץ אותה שוב ושוב.

תוחלת החרטה של זה היא $\mathcal{O}(T)$ (שכן יש $\mathcal{O}(T)$ צעדים בהם בוחרים באקראי). אבל עם בחירה חכמה של ϵ (ספציפית $N = \left(\frac{T}{K}\right)^{\frac{2}{3}}$) אפשר להשיג זמן סאב-לינארי.

שדרוג - טכניקת ϵ -greedy constant

בכל צעד, מגרילים מטבע ממושקל ϵ , ולפי זה בוחרים אם הצעד הזה יהיה exploration או exploitation.

- נשים לב: • גם הצעדים של exploitation עוזרים ל exploration, שכן אם מקבלים reward נמוך מעדכנים את ההסתברות להמשיך לבחור בו.
- לא צריך כאן את T - ולכן זה עובד גם עבור אופק אינסופי.

יש בעיה עם ϵ קבוע - בשלב מסוים אנחנו יודעים מספיק וזה כבר לא רעיון טוב להמשיך לעשות exploration באותה הסתברות כמו בהתחלה.

עוד שדרוג - ϵ קטן עם t

אחרי כל איטרציה של הלולאה, מעדכנים את ϵ . אפשר לעדכן בצורה פשוטה:

$$\epsilon_t \leftarrow \epsilon_{t-1} \cdot \lambda$$

או בצורה קצת יותר מתוחכמת:

$$\epsilon_t \leftarrow \epsilon_{\text{end}} + (\epsilon_{\text{start}} - \epsilon_{\text{end}}) \cdot e^{t \cdot \lambda}$$

פסילת זרועות באמצעות Confidence Bounds

אם אנחנו יודעים שהכי טוב שזרוע מסוימת יכולה לתת פחות טובה מהכי גרוע שזרוע אחרת יכולה לתת - אפשר לפסול את הזרוע הראשונה ולהפסיק לבחור אותה. אלגוריתם UCB1 (Upper Confidence Bound)² מחשב מה הכי טוב שזרוע יכולה לתת - ונבחר לפי זה:

- זרוע שהיא נכונה, תיבחר.
- זרוע שהיא לא נכונה, נבחר אותה אולי בהתחלה - אבל כשהיא תתחיל לקבל תגמולים נמוכים נפסיק לבחור בה.

מעדכנים לפי

$$UCB(a_i) \leftarrow \hat{\mu}(a_i) + \sqrt{\frac{2 \ln t}{n_i}}$$

כאשר המרכיב $\sqrt{\frac{2 \ln t}{n_i}}$ יש בו שני מרכיבים:

$2 \ln t$ ילך ויגדל לפי מספר הצעדים

n_i מספר הפעמים שבחרתי בזרוע הזאת

אם יש זרוע לא טובה, ה- n_i שלה ישאר מאוד קטן אבל $2 \ln t$ שלה יגדל. לכן הרווח-בר-סמך שלה ילך ויגדל, ומתישהו היא תקבל הזדמנות - אבל ככל שנתקדם בצעדים היא תצטרך להתגבר על יותר ויותר.

הregret של האלגוריתם הזה מאוד נמוך - עבור $\mathcal{D}$ על $[0, 1]$ החרטה היא $\mathcal{O}(\sqrt{K \cdot t \cdot \log t})$. גירסת UCB1-Tuned עובדת יותר טוב:

$$UCB(a) = \sqrt{\frac{C \cdot 2 \ln t}{n}} \quad C = \min\left(\frac{1}{4}, V_i(n_i)\right)$$

גירסאות של MAB

- Markovian Bandits - בכל פעם שנכנסים לזרוע state שלה משתנה
- Contextual Bandits - לכל מצב יש MAB משלו
- ∞ -armed Bandits - מרחב הפעולות הוא רציף

²UCB זאת משפחה של אלגוריתמים

דוגמה מהחיים (מחקר אמיתי)

מחקר שנעשה עם חברת ElliQ - מוצר שמציע פעילויות מותאמות למשתמש בבתי אבות.

Enhancing Recommendations using Learning to Prolong Interactions

האופק לא קבוע - האלגוריתם צריך לבחור פעולות (מה להציע למשתמש) שיאריכו את session. בניגוד לאפליקציות כמו נטפליקס או טיק-טוק בהן יש המון אפשרויות, ב-ElliQ הקטלוג יחסית מוגבל (150 אפשרויות) MAB המודל יבחר את מה שאמרו לו הכי הרבה פעמים "כן" - אבל זה לא טוב, כי אז זה יחזור על עצמו והמשתמש ישתעמם. פתרון פשוט (בלי להתעסק במתמטיקה) הוא להשתמש ב-Constrained Multiarmed Bandit - להוסיף כלל שאי אפשר להציע את אותו דבר פעמיים באותו session. אבל אפשר לעשות יותר חכם.

- נגדיר: מרחב הפעולות - הפעילויות שאפשר להציע למשתמש.
- התגמול - בינארי (למשל 1/0 או 5/0)
- מצבים: ב-MAB יש רק מצב אחד, אבל בבעיות אמיתיות יש יותר ממצב אחד. כל פרט שמוסיפים מגדיל את מרחב המצבים באופן אקספוננציאלי. לפעמים עדיף פשוט להתעלם מהמצב ולעשות MAB פשוט.

אלגוריתם Contextual UCB1

הבעיה בד"כ היא לא לדעת מה staten, אלא לדעת מה מתוכו מעניין. אחת מהדרכים לנתח את staten זה לבצע כל מיני חלוקות, ולראות איזה חלוקות (לפי איזה פרמטרים של staten) משפיעות הכי הרבה. רוצים מספר מצבים קטן, כי אז יהיו פחות דוגמאות לכל מצב - שיהיה בעצם MAB בפי עצמו - וזמן הלמידה יהיה יותר גדול.

למה זה עבד?

- אין חלוקה לימים/שעות (שהייתה דורשת יותר נתונים ממה שיש)
- האלגוריתם התבצע לאחר שהחברה כבר עשתה סינון ראשוני לפעילויות.

איך בחרו כמה זמן לחכות לפני שמציעים שוב את אותה פעילות?

עשו exploration first. הניחו שהתגובה של בן אדם היא סיגמואידית - כלומר יש איזהו משך זמן שהתשובה מתחלפת משלילית לפניו לחיובית אחריו מאוד מהר. בשלב ה-exploration למדו את הסיגמואידים האלה.